ASRP: Adaptive Support-Based Response Prediction for Test-Time AI Capability Evaluation

Yuhan Chiu
National Taiwan University

Abstract

Predicting whether an AI system will solve a particular evaluation item is harder than estimating an aggregate benchmark score: item difficulty, subject ability, and the small number of revealed labels all matter. We present **ASRP (Adaptive Support-Based Response Prediction)**, a method for predicting subject–item response probabilities in the Predictive AI Evaluation Competition. ASRP combines a public-data subject and metadata prior, a MiniLM-based amortized Rasch predictor, and a support-conditioned kernel probe. A budget-dependent ensemble combines these branches, followed by conservative topic residual calibration; source-keyed guards are available when a public-source identifier matches. Its streaming acquisition policy uses an adaptive covariance-gain rule with a bounded warm-up phase. We report a public-data proxy evaluation over 5,680 held-out item outcomes from 142 public binary sources and clearly separate it from the hidden Codabench evaluation.

1 Introduction

Modern AI evaluations produce response tables with a binary outcome for each subject–item pair. A benchmark average is useful for ranking systems, but it does not provide a probability for a new item and it can be unstable when only a small subset of items is queried. The Predictive AI Evaluation Competition (PAIEC) therefore asks participants to predict the probability that a subject answers each item correctly while revealing at most 31 labels per subject–benchmark pair.

Our goal is a predictor that uses item text and visible metadata at label budget zero, then adapts to revealed labels without overreacting to a small or unrepresentative support set. We implement **ASRP** as a competition submission rather than as a new psychometric theory.

Relation to item-response modeling. Lord's treatment of item response theory (IRT) models the probability of a correct response as a function of test-taker ability and item difficulty [Lord, 1980]. This is the basis for ASRP's Rasch-style probability: a public subject prior supplies the initial ability, the frozen text model supplies item difficulty, and revealed labels update ability with a penalized Newton–Raphson step.

Content-based and adaptive evaluation. Reliable and Efficient Amortized Model-based Evaluation shows that benchmark averages can be distorted by question difficulty, and proposes amortizing difficulty from question content together with adaptive item selection [Truong et al., 2025]. ASRP uses the same two ideas in the competition setting: MiniLM features predict unseen-item difficulty, while `adaptive_4_warm` implements support selection when the remaining budget permits discretionary choices. ASRP does not use that paper's question generator or evaluation model.

Task-level capability prediction. Agent Psychometrics extends IRT to predict task-level success for agentic coding benchmarks using task features and separate ability factors [Ge et al., 2026].

Preprint.

ASRP follows the same subject–item perspective, but uses the competition's visible metadata, binary outcomes, and a support-conditioned kernel residual instead of an agent/scaffold decomposition.

Capability calibration. *On Calibration of Large Language Models: From Response to Capability* distinguishes confidence that one sampled response is correct from confidence that a model can solve a query overall [Yang et al., 2026]. ASRP targets the latter quantity: $p(x)$ is the probability that the subject answers the current item correctly, and Brier score evaluates this query-level probability. This motivates the conservative calibration layer applied after the Rasch–kernel ensemble.

ASRP has three design choices. First, a frozen MiniLM encoder maps item text to a semantic representation, and a small feed-forward difficulty model maps that representation and text-derived numeric features to an item difficulty. Second, a Rasch-style predictor combines that difficulty with a public-data subject prior. Third, a support-conditioned kernel probe uses acquired item representations and a centered ridge fit to model local residuals. We blend these signals using fixed weights for the six official budgets and apply shrunk topic corrections only after the ensemble.

The current report makes three scope commitments. The public proxy numbers below are not hidden-test results and are not used to infer a Codabench score. The official Empirical Mean is the external baseline used for comparison. Finally, all competition-specific training artifacts are derived from the permitted public measurement-db release; no hidden response table is queried at runtime.

2 Competition setting

At runtime the platform supplies a pair of dictionaries $x = (s, i)$ describing a subject and an item, together with a list of previously acquired binary observations. The predictor returns a finite probability in $[0, 1]$. Eligible subject–benchmark groups have at least 80 distinct items. The platform creates a fixed acquisition/evaluation split and evaluates the same evaluation items at the **six label budgets**

$$n \in \{0, 1, 3, 7, 15, 31\}.$$

Acquisition items are never scored and evaluation outcomes are never revealed. The optional acquisition hook receives the current candidate, acquired labels, and remaining-label context; the interface can also supply a current prediction when requested. ASRP uses the first three inputs.

For a budget n and a subject–benchmark group with N evaluated responses, the Brier score is the mean squared error between the predicted probability p and the binary outcome y:

$$B_n = \frac{1}{N} \sum_{j=1}^{N} (p_j - y_j)^2.$$

The group-level Brier ALC is the normalized area on the logarithmic budget axis and is equivalent to

$$\mathrm{ALC} = 0.1B_0 + 0.2B_1 + 0.2B_3 + 0.2B_7 + 0.2B_{15} + 0.1B_{31}.$$

The official score averages these group-level ALC values equally across subject–benchmark groups. **Lower is better.** All reported local tables use this same weighting.

3 Data and permitted training

The predictor is trained and calibrated from the public `aims-foundations/measurement-db` release and its permitted benchmark inventory [AIMS Foundations, 2026]. The public proxy analysis uses the **142 binary sources** for which the local release contains an eligible group and 40 held-out item outcomes per source, giving 5,680 evaluated rows. The complete source inventory used in that analysis is included in Appendix A; the release itself remains the authoritative source for benchmark metadata and licenses.

The subject prior and metadata statistics are computed from public response aggregates. MiniLM is used as a pretrained encoder; the packaged encoder is `sentence-transformers/all-MiniLM-L6-v2`, based on MiniLM [Wang et al., 2020]. Its weights are **frozen at runtime**. The difficulty model, priors, kernel configuration, calibration statistics, and cluster centroids are prepared offline and bundled with the submission; kernel residual coefficients are fitted online from

revealed labels. The current submission uses **no external LLM or web search API during scoring**. At runtime, the program uses only the supplied subject and item attributes, anonymous benchmark identifiers, and labels revealed within the allowed budget; it does not access hidden evaluation outcomes.

4 ASRP method

Figure 1 gives the complete data flow. The upper lane contains offline artifacts that are fixed before evaluation. The lower lane shows one runtime prediction and the separate label-acquisition loop. Solid arrows show the prediction and acquisition paths; dashed arrows show frozen artifacts and revealed evidence supplied to these paths.

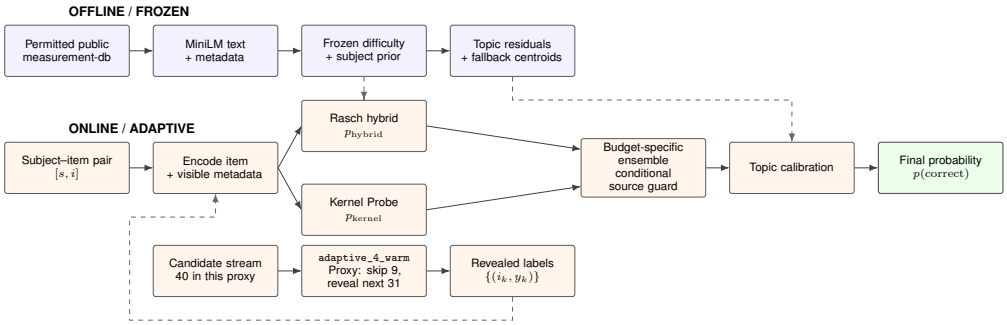


Figure 1: ASRP overview. A pretrained MiniLM encoder is frozen, and permitted public data supply the offline difficulty, prior, and calibration artifacts. At runtime, the Rasch-style hybrid and the support-conditioned kernel probe receive the current subject–item pair and the labels revealed so far. A budget-specific ensemble is followed by topic residual correction. The source guard is conditional on an identifier match and is inactive for this proxy. The acquisition hook supports up to 16 warm-up candidates and subsequent covariance-gain selection; in the reported 40-candidate proxy, the budget constraint instead yields 9 skipped candidates followed by 31 mandatory acquisitions.

4.1 Item representation and amortized difficulty

Let $e_i \in \mathbb{R}^{384}$ be the sentence embedding of the normalized item text from MiniLM. We concatenate e_i with six deterministic numeric text features $z_i \in \mathbb{R}^6$ (including text length, digit proportion, and code or mathematical markers). The **frozen difficulty network** standardizes this vector and applies a three-layer feed-forward map (390–128–32–1):

$$d_i = f_\phi([e_i; z_i]).$$

To compute e_i, the whitespace-normalized text is restricted to its first 6,000 characters and tokenized to at most 256 tokens. Mean pooling and L_2 normalization yield e_i. The hidden layers use ReLU activations; standardized inputs are clipped to $[-10, 10]$ and the resulting difficulty to $[-2, 2]$. This content-based estimate is an amortized substitute for fitting a separate difficulty parameter for every unseen item.

4.2 Public prior and Rasch update

The hybrid uses two public-data priors. The subject-only prior initializes ability, while the metadata prior p_{src} combines the subject's public response rate with visible item-feature statistics for evidence smoothing. For the frozen public subject offset o_s keyed by normalized model name (zero for an unknown name), define

$$m_s = \text{clip}(0.5 + 0.5o_s, 0.05, 0.95), \qquad \theta_0 = \text{logit}(m_s).$$

The Rasch probability is

$$p_{\text{Rasch}}(i) = \sigma(\theta - d_i), \quad \sigma(z) = \frac{1}{1 + e^{-z}}.$$

Here d_i is the frozen content-based item difficulty from Section 4.1. For acquired labels $(y_k, d_k)_{k=1}^n$, ASRP obtains a **maximum a posteriori (MAP) estimate** by maximizing the Bernoulli log-likelihood with a Gaussian prior centered at θ_0:

$$\ell(\theta) = \sum_{k=1}^{n} \log p(y_k \mid \theta, d_k) - \frac{(\theta - \theta_0)^2}{2v}, \qquad v = 2.0.$$

The implementation uses Newton–Raphson iteration with clipped steps. Writing $p_k = \sigma(\theta - d_k)$,

$$g = \sum_k (y_k - p_k) - \frac{\theta - \theta_0}{v}, \qquad h = \sum_k p_k(1 - p_k) + \frac{1}{v}, \qquad \theta \leftarrow \theta + \mathrm{clip}(g/h, -2, 2).$$

It stops when the step magnitude is below 10^{-7} or after 30 iterations, and clips the final ability to $[-12, 12]$. At budget zero, $\theta = \theta_0$.

The metadata prior supplies a smoothed evidence estimate,

$$p_{\mathrm{smooth}} = \frac{S_n + \kappa p_{\mathrm{src}}}{n + \kappa}, \qquad S_n = \sum_k y_k, \quad \kappa = 4.$$

The Rasch and smoothed terms are combined as

$$p_{\mathrm{hybrid}} = (1 - \lambda_n) p_{\mathrm{smooth}} + \lambda_n p_{\mathrm{Rasch}},$$

with frozen $\lambda_n = (0.2875, 0.3000, 0.2875, 0.2750, 0.2250, 0.2375)$ for budgets $(0, 1, 3, 7, 15, 31)$.

4.3 Support-conditioned kernel probe

The **support-conditioned kernel probe** estimates a local correction from the revealed support items. The support contains revealed responses from the current benchmark, potentially spanning multiple subjects. For each support item r, let u_r be its MiniLM vector and v_r its hashed lexical vector, both derived from visible item text, metadata, and interactor descriptions. Unlike the Rasch text encoder, the kernel encoder pools up to four evenly spaced 254-token windows, normalizes each window vector, and normalizes their sum. The lexical vector has 8,192 hashed coordinates. For support size $m > 1$, we center and scale the semantic coordinates and apply inverse-frequency weights to lexical coordinates:

$$\tilde{u}_r = \mathrm{norm}((u_r - \bar{u}) \odot a), \qquad \tilde{v}_r = \mathrm{norm}(v_r \odot b).$$

Here norm denotes L_2 normalization, $\bar{u}$ is the support mean, a is an inverse standard-deviation scale regularized by the median support variance (floored at 10^{-7}), and $b_\ell = \log((1 + m)/(1 + f_\ell)) + 0.1$, where m is the support size and f_ℓ counts support vectors with a nonzero lexical coordinate ℓ. For a single support item, centering and coordinate reweighting are bypassed. For two support items, the similarity kernel is

$$q_{rt} = \frac{\tilde{u}_r^\top \tilde{u}_t + \tilde{v}_r^\top \tilde{v}_t}{2}, \qquad K_{rt} = \exp\left(\frac{\min(q_{rt} - 1, 0)}{\tau}\right),$$

with $\tau = 4 \max(d_{\mathrm{med}}, 10^{-5})$, where d_{med} is the median off-diagonal distance $1 - q_{rt}$ above 10^{-8}, or 0.1 if none qualifies. The query-to-support vector $k(x)$ uses the same transformation and kernel.

The kernel intercept is a two-component Beta mixture. For subject a, let s_a of n_a revealed labels be successes and let o_a be the public offset. Define $m_a = \mathrm{clip}(0.5 + 0.5 o_a, 0.05, 0.95)$. For concentrations $c \in \{1, 8\}$ and prior masses $\pi_1 = 0.3$, $\pi_8 = 0.7$,

$$\alpha_{ac} = m_a c, \quad \beta_{ac} = (1 - m_a)c, \quad \omega_{ac} \propto \pi_c \frac{\mathrm{B}(\alpha_{ac} + s_a, \beta_{ac} + n_a - s_a)}{\mathrm{B}(\alpha_{ac}, \beta_{ac})},$$

$$b_a = \sum_c \omega_{ac} \frac{\alpha_{ac} + s_a}{c + n_a}.$$

Here B is the beta function and the posterior weights are normalized so that $\sum_c \omega_{ac} = 1$. Thus b_a is the conservative subject baseline before using local similarity.

Let M average rows belonging to the same subject, and let $C = I - M$. The centered ridge residual coefficients are

$$\gamma = C(CKC + \lambda I)^{-1} Cy, \qquad \lambda = 0.3,$$

where y contains the revealed binary labels. If $\bar{f}_a$ is the subject mean of the fitted kernel contribution, the query prediction is

$$p_{\text{kernel}}(x, a) = \text{clip}\big(b_a + k(x)^\top \gamma - \bar{f}_a, 0, 1\big) .$$

The kernel contribution is suppressed at budgets 0 and 1 and receives weights $(0, 0, 0.2, 0.4, 1, 1)$ at $(0, 1, 3, 7, 15, 31)$, so a tiny support set cannot override the cold-start prediction. This creates a deliberate budget-dependent division of labor: at low budgets, the Rasch hybrid is the main predictor because its public subject prior and content-based difficulty remain stable when there are too few revealed labels to fit a local kernel residual. The kernel branch is introduced gradually at 3 and 7 labels, then becomes the nominal primary predictor at 15 and 31 labels, giving greater influence to residual corrections fitted from the larger support set. The late-budget source guard can reduce this kernel weight for sources with a regression identified in public diagnostics.

4.4 Topic calibration and source guard

The calibration artifact is generated from public binary responses. ASRP scans the visible metadata in its original order and uses the first topic key–value pair present in the frozen topic-residual table. A topic residual $r_{t,n}$ is shrunk toward zero using its public count m_t:

$$\Delta_{t,n} = \text{clip}\left(\frac{m_t}{m_t + 40} r_{t,n}, -0.06, 0.06 \right) .$$

If no topic key–value pair matches the table, the Rasch encoder's 384-dimensional vector is assigned to the closest of eight frozen centroids by cosine similarity. Cluster residuals use a stronger prior count of 200 and a smaller cap of 0.03. The correction is applied after the budget-specific ensemble.

Public validation identified some sources for which the kernel branch became less accurate at high budgets. At budgets 15 and 31, a matching public-source key supplies a guard $g \in [0, 0.35]$ that multiplies the nominal kernel weight by $1 - g$; the remaining weight goes to the Rasch hybrid. Unmatched identifiers receive zero adjustment. The anonymous aliases in the reproducible proxy do not match these source-name keys, so this guard is inactive in the reported experiment. This guard is a conservative regularizer, not a lookup of hidden test outcomes; source statistics are learned offline from public data.

4.5 Adaptive acquisition

The submitted `labeling.py` implements `adaptive_4_warm`. The policy can skip up to 16 early candidates to establish a local feature geometry, subject to the evaluator's remaining-item constraint. It then estimates the conditional covariance reduction from each candidate using previously revealed items, previously seen candidate features, the current candidate, and the supplied context. The candidate's gain ratio is raised to power four and converted to a probability under the remaining label budget. When the number of remaining candidates equals the remaining label budget, the hook acquires every remaining candidate. The hook never accesses future candidates or evaluation outcomes.

5 Experimental setup

5.1 Public proxy protocol

Evaluation sample. We evaluate on **142 public binary sources**, using one eligible subject selected deterministically in each source and 40 held-out item outcomes per source, for a total of **5,680 evaluation rows**. Each selected subject–source group has at least 80 distinct items. A SHA256 ordering with seed 20261028 selects one subject with at least 80 distinct binary-response items, then selects and orders 80 items: 40 acquisition candidates and 40 evaluation items. For repeated responses to an item, the smallest trial number and then response ID select one recorded binary outcome. The dataset revision, table checksums, subject IDs, item IDs, and response IDs are fixed in the released

split manifest. Evaluation items are separated from the acquisition pool. Here, "held-out" refers to this acquisition/evaluation separation; it does not imply independence from all offline training and calibration data. The proxy reuses public information used in calibration, so its results may be optimistic.

Matched evidence and budgets. The local run uses the packaged `adaptive_4_warm` acquisition hook and a fixed split. With 40 acquisition candidates and a 31-label cap, its warm-up is clipped to $\min(16, 40 - 31) = 9$ skipped candidates. The remaining 31 candidates must all be acquired to meet the cap. Consequently, this proxy exercises the hook but does not measure the benefit of covariance-gain selection. The six budgets $n \in \{0, 1, 3, 7, 15, 31\}$ use nested prefixes of the acquired labels, and each checkpoint scores the same 40 evaluation outcomes per source. **Both predictors receive the same acquired labels and are scored on the same outcomes.** This controls the evidence available to each predictor.

5.2 Baseline and metrics

We use the **Empirical Mean predictor** from the official PAIEC repository [Truong et al., 2026]. For n acquired binary labels in the current subject–benchmark group, it predicts

$$p_{\mathrm{EM}} = \begin{cases} 0.5, & n = 0, \\ \frac{1}{n} \sum_{k=1}^{n} y_k, & n > 0. \end{cases}$$

It uses neither item features nor a learned prior. In this local comparison, we execute its predictor on the same acquisition prefixes supplied to ASRP; the official standalone baseline instead uses the evaluator's default random acquisition policy.

We report Brier scores at all six budgets and Brier ALC with the weights defined in Section 2. Each selected subject–source group contributes equally; because every group has 40 evaluation rows, averaging these group scores is equivalent to averaging over all 5,680 rows. **All results describe one local public-data evaluation**, without repeated-draw confidence intervals. Official formative evaluations draw random subsets from a private test pool; sampling and distribution differences can make their scores better or worse than these local values.

6 Results and analysis

6.1 Public proxy results

Table 1 compares the two predictors on identical evaluation items and acquired labels. ASRP has lower aggregate Brier scores at **all six budgets**. Its Brier ALC is 0.2072, compared with 0.2531 for Empirical Mean, an absolute reduction of 0.0460 (18.2% relative to the baseline). Differences use unrounded scores; negative values favor ASRP. These are local public-proxy results.

Method	B_0	B_1	B_3	B_7	B_{15}	B_{31}	ALC
ASRP	0.2268	0.2245	0.2133	0.1981	0.1939	0.1852	0.2072
Empirical Mean	0.2500	0.3877	0.2505	0.2079	0.1987	0.1918	0.2531
Difference	-0.0232	-0.1632	-0.0372	-0.0098	-0.0048	-0.0066	-0.0460

Table 1: Public proxy Brier scores. Lower is better. ASRP and the official Empirical Mean use the same 5,680-row proxy split (142 sources and 40 held-out items per source). Difference is ASRP minus Empirical Mean. These are local results, not Codabench hidden-test scores.

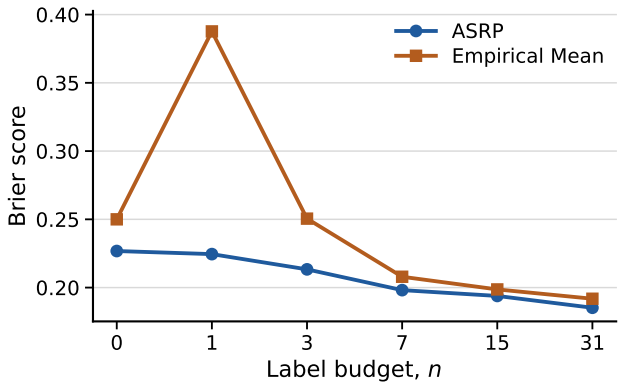


Figure 2: Public proxy learning curves for ASRP and the official Empirical Mean plotted on the official $\log_2(1+n)$ budget axis. Both curves use the same 5,680 public held-out rows as Table 1.

6.2 Analysis across label budgets

Where the improvement occurs. The largest baseline-minus-ASRP gap occurs at **budget 1**: ASRP scores 0.2245 and Empirical Mean scores 0.3877. At budget one, Empirical Mean predicts either zero or one for every query in a group; a single unrepresentative label can therefore produce a large error. ASRP retains its public prior and Rasch component at this budget, moderating such extreme updates. At budget zero, the content- and prior-based ASRP prediction scores 0.2268, compared with the constant baseline's 0.2500.

Behavior as evidence accumulates. ASRP moves from 0.2268 at budget zero to 0.1852 at budget 31. The baseline-minus-ASRP gaps at budgets 15 and 31 are 0.0048 and 0.0066. The budget-dependent design anchors early predictions with the Rasch hybrid and gives the kernel residual more weight as support grows. Topic calibration remains active, but the source-name guard has zero weight adjustment for the anonymous aliases used in this experiment. These results compare the combined predictors conditional on matched evidence; the constrained acquisition trajectory does not measure an active-selection gain.

7 Limitations

Sparse metadata and local adaptation. Some public items lack usable topic metadata, motivating the frozen embedding-cluster fallback. Topic groups with few observations can have residual estimates dominated by sampling noise. This motivates shrinking and capping topic corrections. Likewise, an unrepresentative acquired support set can cause the kernel to over-update. The source guard only limits its weight when a public-source key matches; anonymous or unseen sources do not receive this adjustment. These considerations motivate conservative correction magnitudes; neither topic calibration nor local adaptation guarantees improvement on every source.

8 Reproducibility and disclosure

The release separates inference code in `src/asrp` from frozen artifacts in `artifacts/asrp`. The predictor entry point is `model.py`; `labeling.py` implements the streaming acquisition hook; the frozen artifacts are JSON/NPZ files and the local MiniLM encoder. The complete score reproduction pipeline is run from the repository root with:

```
uv sync --locked
uv run hf auth login
uv run --locked python tools/reproduce.py
```

Users first accept the upstream measurement-db access conditions and authenticate with their own Hugging Face read token. The command downloads four public tables per source from the pinned

revision, verifies their checksums, reconstructs the deterministic split, builds the ASRP submission, runs both predictors at all six budgets, and independently aggregates their per-item errors. It checks the resulting predictions against the published run with a 10^{-7} numerical tolerance. The released frozen weights are used directly; no fitting or parameter selection occurs during this evaluation.

`experiments/proxy_config.json` records the seed and protocol; `experiments/proxy_manifest.json` identifies every selected subject, item, and response. `results/proxy` contains per-item predictions, acquisition decisions, per-source scores, and the aggregate results. `tools/update_report_results.py` generates the table and numeric analysis from the scored output. Task text and metadata are downloaded from the original dataset repository; the released result records contain the selected binary outcomes and model predictions.

The active kernel mixture weights are defined in `src/asrp/model.py`; Rasch mixture weights and runtime settings are recorded in `artifacts/asrp/runtime_config.json`; topic calibration settings and source guards are stored in `artifacts/asrp/topic_calibration.json`. Inference code, frozen artifacts, the locked CPU environment, evaluation scripts, and report source are available at `https://github.com/Hubert0916/arsp`. The official submission checker is distributed separately in the competition starting kit.

The competition-specific artifacts were produced from the permitted public measurement-db inventory. We use the pretrained MiniLM encoder and cite its model family. Codex was used as a coding assistant during development; it is not used as an online predictor in the current submission. No API key is included in the codebase or report.

9 Conclusion

ASRP provides a practical, budget-aware predictor for subject–item capability evaluation. Its main engineering principle is to use content-based and psychometric structure at cold start, use support-conditioned residuals only when evidence is available, and shrink topic corrections and late-budget updates toward conservative defaults. In the local public proxy, ASRP achieves lower aggregate Brier scores than Empirical Mean at all six budgets.

A Public source inventory

The following public binary source identifiers occur in the 5,680-row proxy analysis. They identify the evaluation sources; the benchmark inventory and upstream licenses remain authoritative.

```
abstract_reason
adaptivestep
aegis
afrieval
afrimedqa
agentic_review_perturbation
agi_elo
ai2d_test
algotune
alignment_faking
alpha_sql
appworld
averimatec
babilong
bbq
beavertails
benger
bertaqa
bird_sql
brace
braveguard
bridging_gap
causal_axioms
cellverse
ceval
clasheval
cmmlu
critic_discernment_game
crow
cruxeval
dave
```

devbench
disco
editbench
edu_circuit_hw
egoschema
elicitation_game
emoji_attack
engdesign
enginemt_qa
erbench
evouna
exploitgym
faithcot
felm
fewshot_ttt_bbh
financebench
finegrain_t2i
hallusionbench
harmbench
healthadminbench
helm_afr
helm_bbq
helm_thaiexam
hle
icpc2_code_selector
igakuqa
igakuqa119
ikp
interchangeable_token_embeddings
jailbreakbench
judgetuning
kernelbench
kmmlu
kormedmcqa
legal_rag_bench
legalbench
lingoly
liveaopsbench
llm_bp_tag
llm_uncertainty_bench
llmail_inject
llms_lean_formalization
mathconstraint
mathvista_mini
medhelm_ehr_sql
medhelm_head_qa
medhelm_med_mcqa
medhelm_med_qa
medhelm_medbullets
medhelm_medcalc_bench
medhelm_medec
medhelm_medhallu
medhelm_pubmed_qa
medhelm_race_based_med
mind2web
mj_bench
mmbench_v11
mmlu
mmlupro
monkey_power_laws
morphkv
mtbbench
multi_swebench
naep_llm_students
naturebench
nester
nyu_ctf_bench
ood_prediction
openbiorq
os_harm
perfcodebench
perumedqa
pku_saferlhf
planbench
proofaug
prox
pxplore
real_webagents
realtimeqa
reliancescope
researchcodebench

revolve
risebench
robust_reasoning_benchmark
safeagentbench
sgrades
sib200
sketchjudge
software_world_models
statqa
subjective_qa
sugarcrepe
sustainable_food
swe_rebench
swe_smith
swebench_live
swepolybench
tau2_bench
taubench
tensortrust
truthfulqa_mc
tulu_human_eval
tumlu
visual_riddles
visualwebarena
vl_rewardbench
wikihow_agent
wmt_mqm
workarena
workbench_revisited
worldcentralbanks

References

Frederic M. Lord. *Applications of Item Response Theory to Practical Testing Problems*. Lawrence Erlbaum Associates, 1980.

Sang Truong, Yuheng Tu, Percy Liang, Bo Li, and Sanmi Koyejo. Reliable and efficient amortized model-based evaluation. *arXiv preprint arXiv:2503.13335*, 2025. URL https://arxiv.org/abs/2503.13335.

Chris Ge, Daria Kryvosheieva, Daniel Fried, Uzay Girit, and Kaivalya Hariharan. Agent psychometrics: Task-level performance prediction in agentic coding benchmarks. In *Conference on Language Modeling*, 2026. URL https://arxiv.org/abs/2604.00594.

Sin-Han Yang, Cheng-Kuang Wu, Chieh-Yen Lin, Yun-Nung Chen, Hung-yi Lee, and Shao-Hua Sun. On calibration of large language models: From response to capability. *arXiv preprint arXiv:2602.13540*, 2026. URL https://arxiv.org/abs/2602.13540.

AIMS Foundations. measurement-db: Public benchmark inventory and response data. Hugging Face dataset, 2026. URL https://huggingface.co/datasets/aims-foundations/measurement-db.

Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. In *Advances in Neural Information Processing Systems*, 2020. URL https://arxiv.org/abs/2002.10957.

Sang Truong, Wale Salaudeen, Nhi Truong, Serena Wang, Angelina Wang, Luke Guerdan, Ziang Xiao, Xing Xie, Nick Haber, and Sanmi Koyejo. PAIEC predictor baselines. GitHub repository, 2026. URL https://github.com/aims-foundations/paiec_baseline.